

Fidenz - Assignment (Full Stack)

Objective

Develop a secure weather analytics application that retrieves weather data, processes it using a custom metric you design, and presents meaningful insights.

Your solution must include:

- Weather data retrieval
- Data processing using a custom Comfort Index (designed by you)
- Responsive UI
- Caching
- Authentication and Authorization (Auth0)

Part 1: Weather Analytics Application (6 Hours)

This part tests your problem-solving, data processing, and UI implementation skills.

Task Overview

Build a web/API application that does the following:

- Reads city codes from [cities.json](#)
- Fetches weather data from OpenWeatherMap
- Register on [OpenWeatherMap](#) to obtain an API key
- Use the CityCode values to fetch weather data from OpenWeatherMap's API
- Computes a Comfort Index Score for each city (formula defined by you)
- Displays weather + score + ranking
- Provides server-side caching
- Supports desktop + mobile layouts

Design Your Own Comfort Index Algorithm

You must create a custom "Comfort Index Score" using at least three parameters, such as:

- Temperature
- Humidity
- Wind Speed
- Cloudiness
- Pressure

- Visibility
- Dew Point (Not directly available in response)

Rules:

- Your comfort score must be a numerical value from 0 to 100.
- You must include a short explanation why you chose your formula, placed inside your README.
- The backend must compute this score, not the frontend.
- Cities must be ranked from "Most Comfortable" to "Least Comfortable."

Detailed Steps

Step 1: Extract City Codes

- Download and parse [cities.json](#).
- Extract CityCode values into an array.
- Minimum 10 cities must be processed.

Step 2: Fetch Weather Data

Use OpenWeatherMap's API:

GET <https://api.openweathermap.org/data/2.5/weather?id={city id}&appid={API key}>

Example:

<https://api.openweathermap.org/data/2.5/weather?id=2172797&appid=439d4b804bc8187953eb36d2a8c26a02>

Sample Response:

```
{
  "coord": {
    "lon": 145.7667,
    "lat": -16.9167
  },
  "weather": [
    {
      "id": 804,
      "main": "Clouds",
      "description": "overcast",
      "icon": "04n"
    }
  ],
  "base": "stations",
  "main": {
    "temp": 299.12,
    "feels_like": 299.12,
    "temp_min": 299.12,
    "temp_max": 299.12,
    "pressure": 1012,
    "humidity": 73,
    "sea_level": 1012,
    "grnd_level": 992
  },
  "visibility": 10000,
  "wind": {
    "speed": 2.57,
    "deg": 180
  },
  "clouds": {
    "all": 100
  },
  "dt": 1763642260,
  "sys": {
    "type": 1,
    "id": 9490,
    "country": "AU",
    "sunrise": 1763580841,
    "sunset": 1763627476,
    "timezone": 36000,
    "id": 2172797,
    "name": "Cairns",
    "cod": 200
  }
}
```

Step 3: Compute the Comfort Index Score

You must:

- Define your formula
- Explain your reasoning in the README
- Apply the formula to each city
- Produce a score from 0 to 100
- Produce a sorted list of cities based on the score

Step 4: Display Weather Information

For each city, display:

- City Name
- Weather Description
- Temperature
- Your Comfort Score
- Rank Position

Step 5: Implement Server-Side Caching

- Cache raw weather API responses for 5 minutes
- Cache the processed output separately (optional but preferred)
- Include a debug endpoint to show cache status (HIT / MISS)

Step 6: Responsive UI

- Must support both mobile and desktop

Part 2: Authentication & Authorization (3 Hours)

Step 1: Implement Authentication (Auth0)

- Only authenticated users can access Comfort Index dashboard
- Implement login/logout flow

Step 2: Multi-Factor Authentication

- Enable MFA via email verification

Step 3: Restrict Signups

- Disable public signups
- Only whitelisted users can log in

Create a test user:

Email: careers@fidenz.com

Password: Pass#fidenz

Additional Requirements

Technology

You may use any stack (React, Vue, Angular, Node, .NET, Python, etc.)

Repository

Push to GitHub or GitLab. Grant access to:

- kanishka.d@fidenz.com
- srimal.w@fidenz.com
- narada.a@fidenz.com
- amindu.l@fidenz.com
- niroshanan.s@fidenz.com

Documentation

Your README must include:

- Setup instructions
- Explanation of your Comfort Index formula
- Reasoning behind variable weights
- Trade-offs you considered
- Cache design explanation
- Any known limitations

Bonus (Not mandatory but rewarded)

- Dark mode
- Unit tests for the Comfort Index function
- Sorting/filtering on frontend
- Graphs (temperature trend per city)

Technologies & References

To complete this assignment, you may use any relevant technologies and libraries. Useful references:

- Core Concepts
- HTTP, Sessions, Authentication & Authorization
- [Auth0 Authentication \(Docs\)](#)
- JWT Tokens, Single Sign-On (SSO), Multi-Factor Authentication (MFA)
 - [OpenID Connect Flows](#)
 - [JWT Introduction](#)
 - [Single Sign-On \(SSO\)](#)
 - [Auth0 MFA](#)

All code must be authored by you, and you are expected to have a deep understanding of both your implementation and the underlying concepts. By the end of the assignment, you should be fully able to explain your design decisions, discuss the technologies involved, and reason about the architecture. If your assignment is selected, the evaluation interview will include an in-depth discussion of your solution as well as a live code modification to verify your understanding.

Part 3: Screen Recording (Required)

In addition to the deliverables above, submit one unedited screen recording, 5 to 7 minutes long. No recording means your assignment will not be reviewed.

In the recording, you must:

1. Explain one key design decision from your solution. Pick something with real trade-offs behind it, for example why you weighted the Comfort Index parameters the way you did, or how you structured your caching layer. Walk through your reasoning as if explaining it to a teammate who has not seen the code.
2. Live-implement the following small extension, recording your screen the whole time: add one additional parameter to your existing Comfort Index formula (for example, incorporate Visibility or Pressure if you have not already used it), recompute the scores, and confirm the city ranking updates correctly. Do this without pausing the recording or pre-writing the change beforehand. Talk through what you are doing as you do it.

Recording requirements

- Must be a single, unedited take. Do not cut, splice, or record multiple takes and submit the best one.
- Keep it to 5-7 minutes. Going over is fine by a minute or two, but plan to be concise.
- Your voice must be audible throughout.

Deliverables and Submission

- Upload your screen recording to Google Drive and set sharing to "Anyone with the link" so it can be viewed without a request being sent.
- Push your code to GitHub or GitLab as described above, with access granted to the reviewers listed.
- Within 7 days of receiving this assignment, reply to this email with your repository link and your Google Drive video link.
- Before sending, open the video link in an incognito or private browser window to confirm it is actually accessible without logging in.